

אלגוריתמי ניווט מיפוי

ושערוך מיקום

תרגיל 2

Epipolar Visual Odometry להערכת מסלול רחפן

-

תיאור המשימה

במשימה זו נבנה מערכת לניתוח
רצף תמונות מרחפן, המבוססת על
אלגוריתם Visual Odometry/SLAM.

המערכת תעריך את תנועת
המצלמה במרחב (3D space) על ידי
זיהוי והתאמת תכונות בין תמונות
רצופות.

המסלול של המצלמה יוצג בזמן
אמת באמצעות ספריית Pangolin,
ללא צורך ביצירת מפת הסביבה.
הפלט העיקרי הוא ויזואליזציה של
המסלול המוערך, תוך שיקוף
תנועות הרחפן בזמן אמת.

מבנה התרגיל והקבצים

במסגרת התרגיל מועבר קובץ בשם
Dataset_VO.zip, המכיל רצף סדרתי
של תמונות שנלקחו מרחפן.
התמונות ממוינות בסדר כרונולוגי,
כאשר כל תמונה היא תוצאה
מתזוזה קטנה יחסית של המצלמה
לעומת התמונה הקודמת.
המשימה כוללת פיתוח מערכת
לניתוח הרצף, תוך שימוש
באלגוריתם Visual Odometry

להערכת תנועת המצלמה במרחב,
והצגת המסלול בזמן אמת.

קבצי קוד: קבצי ++C עם שימוש ב-
Pangolin, OpenCV (לטעינת תמונות,
תכנות תכונות וקטוריות), ובמחלקות
מוגדרות היטב.

או קבצי פייתון עם שימוש ב
pangolin, OpenGL, nummypy, cv2

1- טעינת הנתונים:

יש ליצור מחלקה בשם `FRAME`
שמייצגת תמונה בודדת ברצף.
המחלקה תכלול את המשתנים
והמאפיינים הבאים:

- `KeyPoints` – אוסף נקודות מפתח
(כגון SIFT, ORB וכו') שנמצאו
בתמונה.

- `Descriptors` – תכונות תואמות
(תיאוריות) של נקודות המפתח.

- `Id` – מזהה ייחודי לתמונה
(לדוגמה, מספר סידורי).

- `Pose` – מצב היעד של המצלמה
בזמן צילום התמונה (כולל מיקום
וכיוון, ממומש כ-`Eigen::Matrix4d` או
מבנה דומה).

** אפשר להוסיף משתנים נוספים
לפי הצורך **, כגון:

- `Image` – התמונה עצמה או נתוני
תצוגה.

- `Timestamp` – תזמון הצילום.

- 'Processed' – סמן האם התמונה עברה עיבוד.

2- הגדרת מיקום הרחפן:

פרמטרי המיקום

RotationMatrix: מטריצת סיבוב (3×3)
המייצגת את הסיבוב של המערכת.

TranslationVector: וקטור תזוזה (3)
המייצגת את התנועה של המערכת.

מבנה Pose

במבנה Pose שדורג קודם, יש להוסיף
שדות RotationMatrix ו-
TranslationVector לאחסן את
הפרמטרים הללו.

3- היחס בין מערכות הצירים:

אפשר לבחור כל מערכת צירים, אין
הגבלה בבחירת כיווני מערכת
הצירים.

המערכת המועדפת:

Z - קדימה/אחורה

X - ימינה/שמאלה

Y - למעלה/למטה

4- חישוב תנועה ייחסית בין תמונות:

בצע חישוב תנועה ייחסית בין שתי
תמונות באמצעות שימוש בגיאוטריה
אפיפולארית

5- מטריצה פנימית:

בשביל לפשט את המשימה, לא
צריך לבצע קלברציה ואפשר

להשתמש בממדי התמונה בשביל לייצר מטריצה פנימית

6- ייצוא תוצאות:

עליכם להגיש קוד שלאחר הרצתו,
אמור להופיע שני חלונות:

החלון הראשון יציג את התמונה
הנוכחית עם נקודות הקי פוינט (Keypoints).

החלון השני יציג את המסלול
שנמצא לפי התקדמות התמונות.
אפשר לייצר פריים שמדמה מצלמה
באמצעות ספרייט Pangolin, כדי
להמחיש באופן ויזואלי את התהליך
בזמן הריצה.

שפת תכנות והנחיות נוספות:
ניתן לכתוב את הקוד בפייתון/C++.

שמות המשתנים והפונקציות צריכים
להיות בשמות קטנים, בשימוש
במיילים, מופרדים באמצעות קו
תחתון (_).

שמות המחלקות או הקבצים יכולים
להתחיל באות גדולה.

**הקפידו על קריאות ותחביר ברור
(למשל, PEP8 לפייתון).**

בהצלחה